

Homework 7 Report

Wilson Neira

Part II: The (Simulated!) Problem

Phase 1: Build Your Current System

HW7 > order-system > src >  main.go > ...

```
19
20 type Item struct {
21     ItemID    int    `json:"item_id"`
22     Name      string `json:"name"`
23     Quantity  int    `json:"quantity"`
24 }
25
26 type Order struct {
27     OrderID    string    `json:"order_id"`
28     CustomerID int       `json:"customer_id"`
29     Status     string    `json:"status"` // pending, processing
30     Items      []Item    `json:"items"`
31     CreatedAt  time.Time `json:"created_at"`
32 }
```

HW7 > order-system > src >  Dockerfile > ...

```
1 FROM golang:1.24-alpine AS build
2 WORKDIR /app
3 COPY . .
4 RUN go mod init order-system || true
5 RUN go mod tidy
6 RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -o server
7
8 FROM alpine:latest
9 WORKDIR /app
10 COPY --from=build /app/server .
11 EXPOSE 8080
12 CMD ["/server"]
```

```
go mod init order-system
go mod tidy
go run main.go
```

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems> curl http://localhost:8080/health
```

```
StatusCode      : 200
StatusDescription : OK
Content         : {"status":"ok"}
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 15
                  Content-Type: text/plain; charset=utf-8
                  Date: Sat, 14 Mar 2026 22:32:16 GMT

                  {"status":"ok"}
Forms           : {}
Headers         : [[Content-Length, 15], [Content-Type, text/plain; charset=utf-8], [Date, Sat, 14 Mar 2026 22:32:16 GMT]]
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 15
```

Test Configuration (use Locust):

 Dockerfile	U	13	class SyncOrderUser(HttpUser):
 go.mod	U	14	wait_time = between(0.1, 0.5)
 go.sum	U	15	
 main.go	U	16	@task
> terraform	●	17	def create_sync_order(self):
> tests	●	18	self.client.post("/orders/sync", json=make_order(),
> __pycache__	●		
 locust_async.py	U		
 locust_sync.py	U		
 README.MD	U		
 terraform.tfstate	U		

```
12
13 variable "vpc_cidr" {
14 |   default = "10.0.0.0/16"
15 }
16
17 variable "public_subnet_1" {
18 |   default = "10.0.1.0/24"
19 }
20
21 variable "public_subnet_2" {
22 |   default = "10.0.2.0/24"
23 }
24
25 variable "private_subnet_1" {
26 |   default = "10.0.10.0/24"
```

```

resource "aws_security_group" "ecs" {
  name     = "${var.project_name}-ecs-sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    from_port     = 8080
    to_port       = 8080
    protocol      = "tcp"
    security_groups = [aws_security_group.alb.id]
  }
}

```

HW7 > order-system > terraform >  sns_sqs.tf

```

1 resource "aws_sns_topic" "orders" {
2   name = "order-processing-events"
3 }
4
5 resource "aws_sqs_queue" "orders" {
6   name                        = "order-processing-queue"
7   visibility_timeout_seconds = 30
8   message_retention_seconds  = 345600
9   receive_wait_time_seconds  = 20
10 }
11
12 data "aws_iam_policy_document" "sqs_policy" {
13   statement {
14     effect = "Allow"

```

```
54 resource "aws_ecs_task_definition" "processor" {
63   container_definitions = jsonencode([
68
69     portMappings = [{
70       containerPort = 8080
71       hostPort      = 8080
72       protocol      = "tcp"
73     }]
74
75     environment = [
76       { name = "APP_MODE", value = "processor" },
77       { name = "AWS_REGION", value = var.aws_region },
78       { name = "SQS_QUEUE_URL", value = aws_sqs_queue.orders
79       { name = "WORKERS", value = "1" },
80       { name = "PORT", value = "8080" }
81     ]
1
```

HW7 > order-system > terraform >  outputs.tf

```
1  output "alb_dns_name" {  
2    value = aws_lb.main.dns_name  
3  }  
4  
5  output "sns_topic_arn" {  
6    value = aws_sns_topic.orders.arn  
7  }  
8  
9  output "sqs_queue_url" {  
10   value = aws_sqs_queue.orders.id  
11 }  
12  
13 output "ecs_cluster_name" {  
14   value = aws_ecs_cluster.main.name  
15 }
```

docker build -t ordersys .

aws configure

aws ecr create-repository --repository-name ordersys --region us-east-1

aws ecr get-login-password --region us-east-1 | docker login --username AWS
--password-stdin 316009999564.dkr.ecr.us-east-1.amazonaws.com

docker tag ordersys:latest

316009999564.dkr.ecr.us-east-1.amazonaws.com/ordersys:latest

docker push 316009999564.dkr.ecr.us-east-1.amazonaws.com/ordersys:latest

terraform init

terraform apply -auto-approve

-var="container_image=316009999564.dkr.ecr.us-east-1.amazonaws.com/ordersys:latest"

terraform output -raw alb_dns_name

ordersys-alb-56403289.us-east-1.elb.amazonaws.com

Test Health

curl <http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com/health>

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW7\order-system\terraform> curl http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com/health
```

```
StatusCode      : 200
StatusDescription : OK
Content         : {"status":"ok"}
RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Content-Length: 15
                  Content-Type: text/plain; charset=utf-8
                  Date: Sun, 15 Mar 2026 00:10:38 GMT

                  {"status":"ok"}
Forms           : {}
Headers         : {[Connection, keep-alive], [Content-Length, 15], [Content-Type, text/plain; charset=utf-8], [Date, Sun, 15 Mar 2026 00:10:38 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 15
```

Test Sync Endpoint Once

Invoke-RestMethod -Method POST -Uri

"http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com/orders/sync"

-ContentType "application/json" -Body (@{ customer_id = 123; items = @(@{ item_id = 1; name = "shirt"; quantity = 1 }) } | ConvertTo-Json -Depth 5)

```
● PS C:\Users\Owner\Documents\Audacity\Building-Scalabl
  http://ordersys-alb-56403289.us-east-1.elb.amazonaws.c
  { item_id = 1; name = "shirt"; quantity = 1 }) } | Co
```

```
order_id      : 1773533723623979621
customer_id   : 123
status        : completed
items         : {@{item_id=1; name=shirt; quantity=1}}
created_at    : 2026-03-15T00:15:23.623980772Z
```

Locust

locust -f locust_sync.py --host

<http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com>

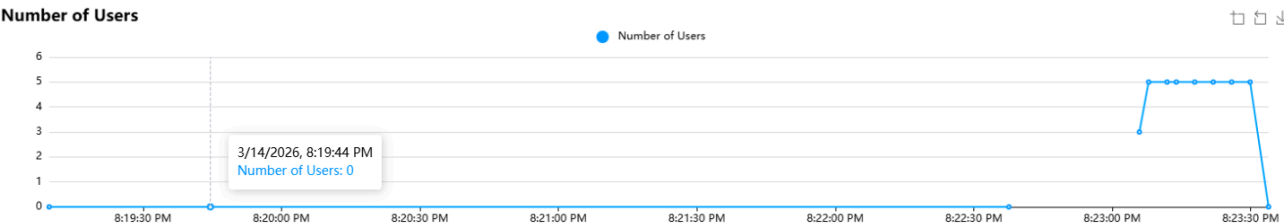
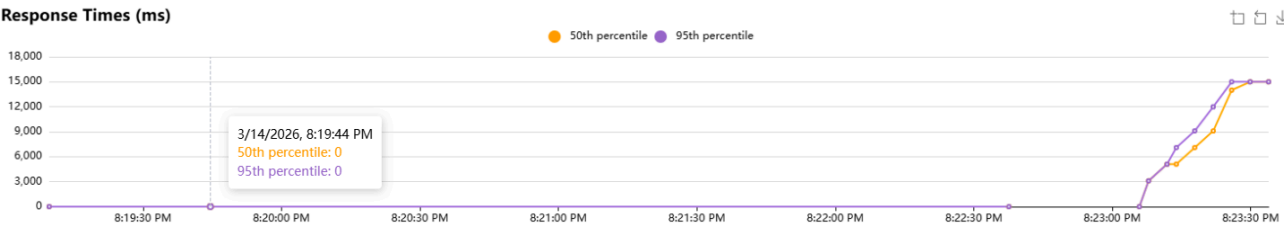
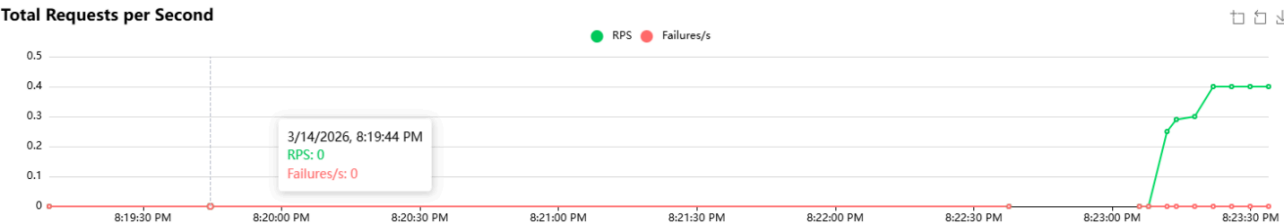
Test Normal Operations

spawn rate: 1 user/second, test endpoint: POST /orders/sync, wait time: random 100–500ms

5 concurrent users, Spawn rate 1, 30 seconds

The synchronous system returned 0 HTTP failures during the 30-second normal-load test, but performance was already poor: only 9 requests completed, median latency was 12,000 ms, and average latency was 10,528.13 ms.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/sync	9	0	12000	15000	15000	10528.13	3102	14896	212.78	0.4	0
	Aggregated	9	0	12000	15000	15000	10528.13	3102	14896	212.78	0.4	0



Services

Tasks

Infrastructure

Metrics

Scheduled tasks

Configuration

Event history

Tags

Tasks (2)

Last updated
March 14, 2026, 20:26 (UTC-4:00)

Manage tags

Stop

Run new task

Filter tasks by property or value

Filter desired status
Any desired status

Filter launch type
Any launch type

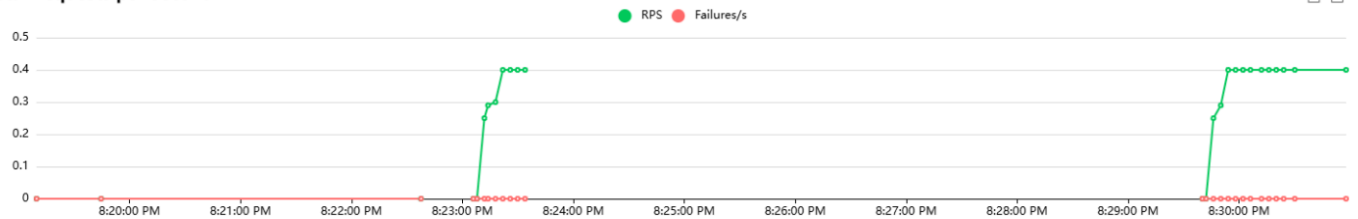
< 1 > ⚙

☐	Task	Last status	Desired status	Task definition	Health status	Created at	Started by	Started at
☐	 f5d9aa9147da42c5ab970...	 Running	 Running	ordersys-api:1	 Unknown	22 minutes ago	ecs-svc/69455736819...	21 minut
☐	 f6a936c562dd41838924a...	 Running	 Running	ordersys-processor:1	 Unknown	24 minutes ago	ecs-svc/39435350011...	21 minut

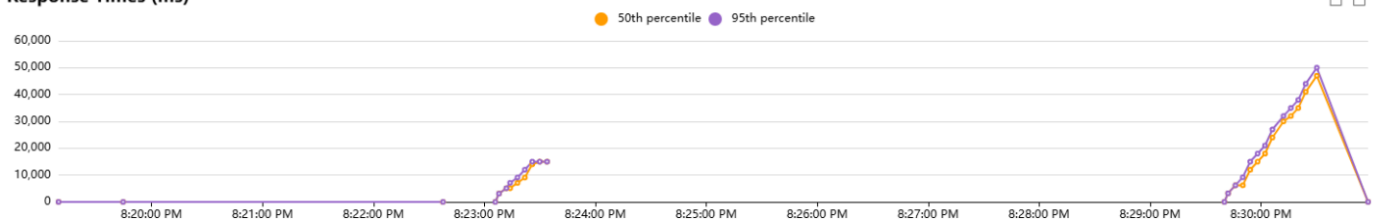
flash sale spawn rate: 10 users/second, 20 concurrent users, 60 seconds, endpoint: POST /orders/sync

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/sync	19	0	30000	56000	56000	29693.51	3159	56181	212.95	0.4	0
	Aggregated	19	0	30000	56000	56000	29693.51	3159	56181	212.95	0.4	0

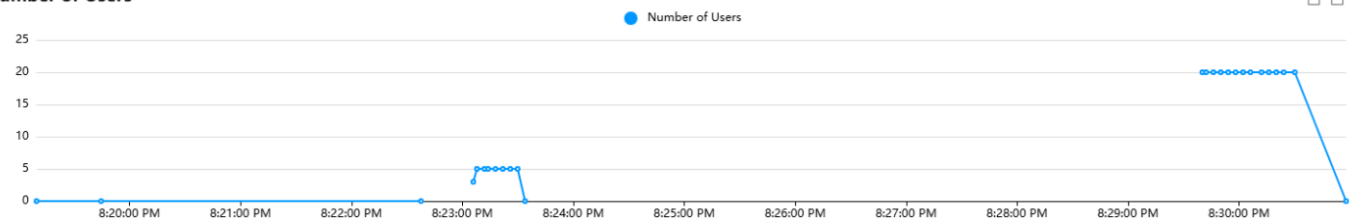
Total Requests per Second



Response Times (ms)



Number of Users



What happens to your customers?

Requests pile up because only one payment can be processed every 3 seconds. In the flash-sale Locust run, only 19 synchronous requests completed in 60 seconds, with 0 HTTP failures but extreme latency: median response time was 30,000 ms, average response time was 29,693.51 ms, and the maximum response time reached 56,181 ms. Customers would experience very long waits for confirmation, and many real users would likely abandon the request even if the server eventually returned 200 OK.

Phase 2: Analyze the Bottleneck

Do the math:

Payment processor speed: 1 order per 3 seconds

With 20 concurrent customers: Maximum throughput = _ orders/second

Flash sale demand: 20 orders/second

Orders lost per second: _

The harsh reality: You can't make payment processing faster, so what CAN you change?

Payment speed:

1 order / 3 seconds = 0.33 orders per second

With 20 concurrent customers, maximum throughput is still:

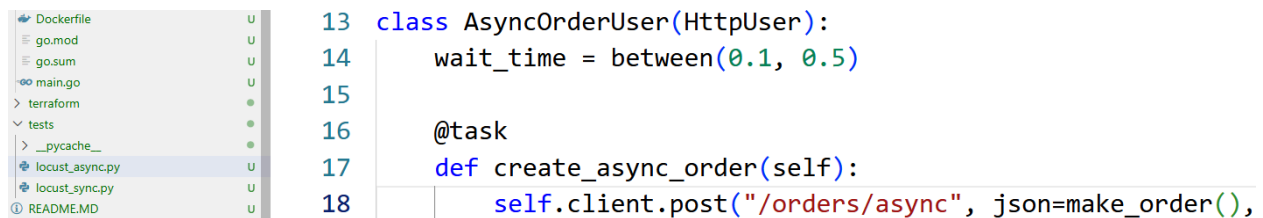
0.33 orders/second

If flash sale demand is treated as 20 orders/second:

Orders lost per second = 20 - 0.33 = 19.67

The system bottleneck is 0.33/sec.

Phase 3: The Async Solution



locust -f locust_async.py --host

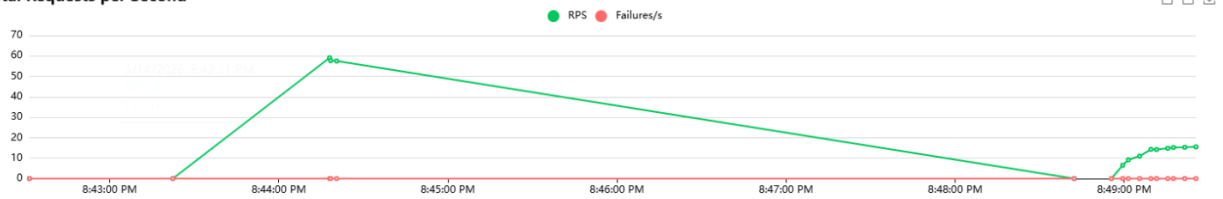
<http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com>

Number of users: 5, Spawn rate: 1, Run time: 30s

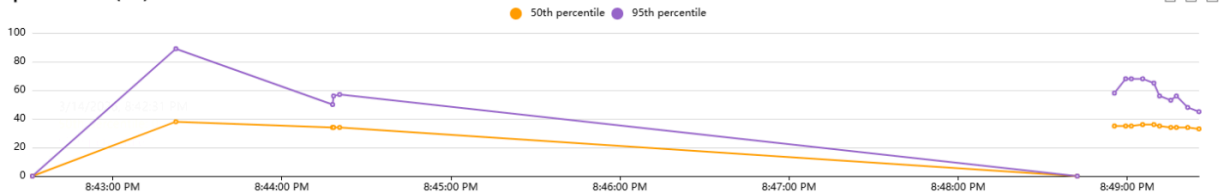
Under normal async load, the system handled 425 requests with 0 failures, median latency of 34 ms, and average latency of 37.71 ms.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	425	0	34	56	100	37.71	23	205	81	15.5	0
	Aggregated	425	0	34	56	100	37.71	23	205	81	15.5	0

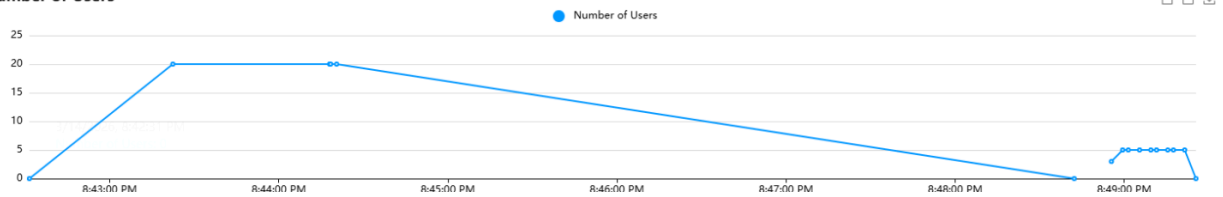
Total Requests per Second



Response Times (ms)



Number of Users

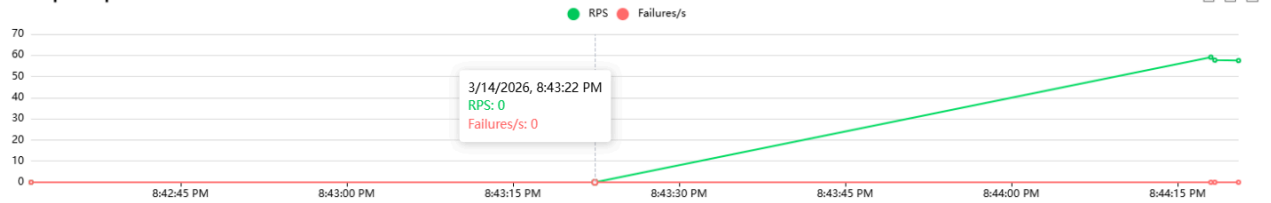


Number of users: 20, Spawn rate: 10, Run time: 60s

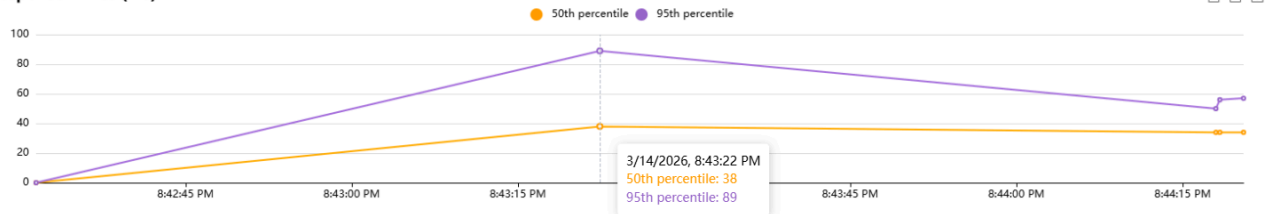
Under flash-sale async load, the system handled 3,540 requests with 0 failures, median latency of 34 ms, average latency of 38.07 ms, and observed throughput near 57.5 requests per second. This confirms that the async endpoint accepted traffic quickly while shifting the real bottleneck to background processing.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3540	0	34	55	95	38.07	23	384	81	57.5	0
	Aggregated	3540	0	34	55	95	38.07	23	384	81	57.5	0

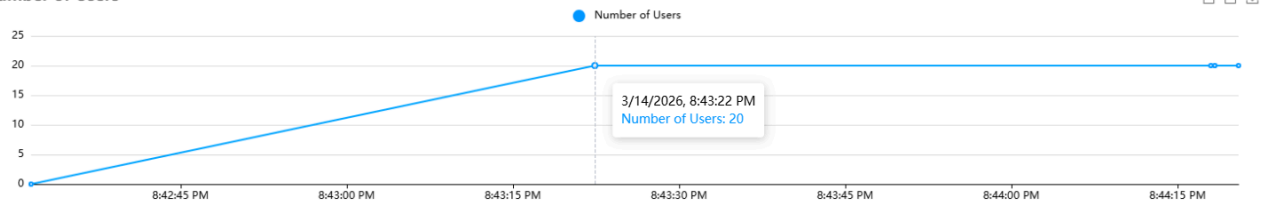
Total Requests per Second



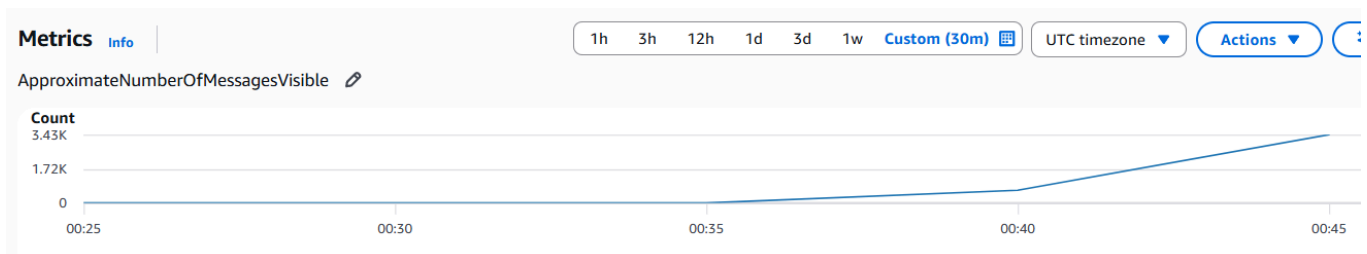
Response Times (ms)



Number of Users



Phase 4: The Queue Problem



Analysis:

Observed async flash-sale acceptance rate: 3,540 requests in 60 seconds, or about 59 requests/second. ~60/second

Single worker processing rate: 1 order per 3 seconds = 0.33/second

Queue growth rate $\approx 59 - 0.33 = 58.67$ messages/second.

Time to clear backlog ≈ 176 minutes, based on an estimated backlog of about 3,520 messages after a 60-second flash-sale run.

Do the theoretical queue math

With 1 worker goroutine:

Processing rate = $1 / 3 = 0.33$ orders/sec

If acceptance rate is about 60/sec:

Queue growth rate = $60 - 0.33 = 59.67$ messages/sec

If the flash sale lasts 60 seconds:

Backlog after test = $59.67 \times 60 = 3580.2$ messages

Time to clear backlog with 1 worker:

$3580.2 / 0.33 = 10849$ seconds

$10849 / 60 = 180.8$ minutes

So about:

181 minutes

If you instead use the 20/sec version:

Queue growth rate = $20 - 0.33 = 19.67$ /sec

Backlog after 60s = 1180.2 messages

Clear time = $1180.2 / 0.33 = 3576$ seconds ≈ 59.6 minutes

Phase 5: Scale Your Workers

1 worker to 5 goroutines

HW7 > order-system > terraform >  ecs_services.tf

```
54 resource "aws_ecs_task_definition" "processor" {
63     container_definitions = jsonencode([
75         environment = [
76             { name = "APP_MODE", value = "processor" },
77             { name = "AWS_REGION", value = var.aws_region },
78             { name = "SQS_QUEUE_URL", value = aws_sqs_queue.orders
79             { name = "WORKERS", value = "5" },
80             { name = "PORT", value = "8080" }
81     ]
```

terraform apply -auto-approve

-var="container_image=316009999564.dkr.ecr.us-east-1.amazonaws.com/ordersys:latest"

locust -f locust_async.py --host

<http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com>

users: 20, spawn rate: 10, run time: 60s, Rate for 5 workers $5/3 = 1.67$ orders/sec

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3579	0	32	53	93	35.27	22	287	81	60.8	0
	Aggregated	3579	0	32	53	93	35.27	22	287	81	60.8	0



5 worker to 20 goroutines

```
{ name = "WORKERS", value = "20" },
```

terraform apply -auto-approve

-var="container_image=316009999564.dkr.ecr.us-east-1.amazonaws.com/ordersys:latest"

locust -f locust_async.py --host

<http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com>

users 20, spawn rate 10, run time 60s, Rate for 20 workers $20/3 = 6.67$ orders/sec

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3565	0	34	53	87	36.62	23	186	81	59.2	0
	Aggregated	3565	0	34	53	87	36.62	23	186	81	59.2	0



20 worker to 100 goroutines

```
{ name = "WORKERS", value = "100" },
```

terraform apply -auto-approve

-var="container_image=316009999564.dkr.ecr.us-east-1.amazonaws.com/ordersys:latest"

locust -f locust_async.py --host

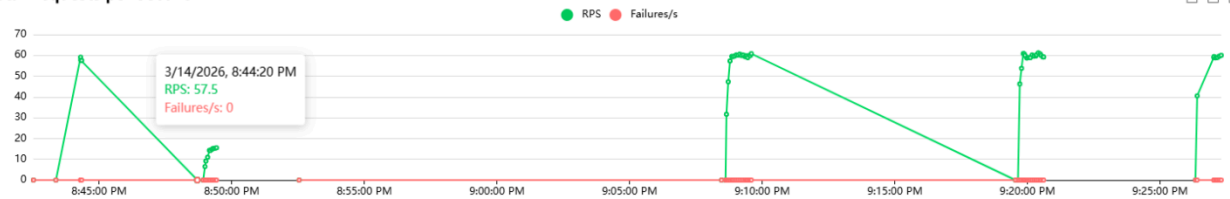
<http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com>

users 20, spawn rate 10, run time 60s, Rate for 100 workers $100 / 3 = 33.33$

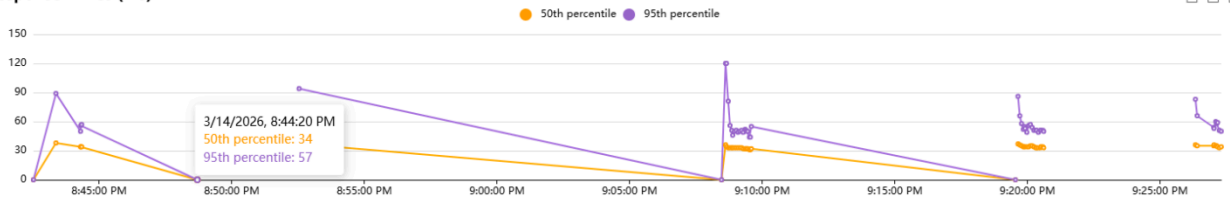
orders/sec

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/async	3546	0	34	56	91	37.36	22	155	81	60	0
	Aggregated	3546	0	34	56	91	37.36	22	155	81	60	0

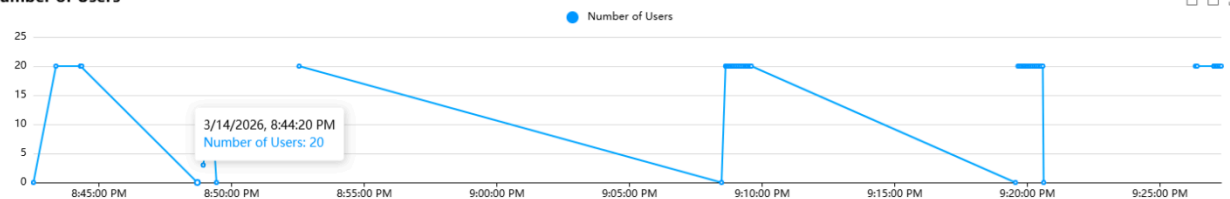
Total Requests per Second



Response Times (ms)



Number of Users



Find the balance: What's the minimum workers needed to prevent queue buildup at 60 orders/second?

To keep up with 60 orders/sec:

Each worker handles:

$$1/3 = 0.33 \text{ orders/sec}$$

Needed workers:

$$60/0.33 \approx 180$$

More exactly:

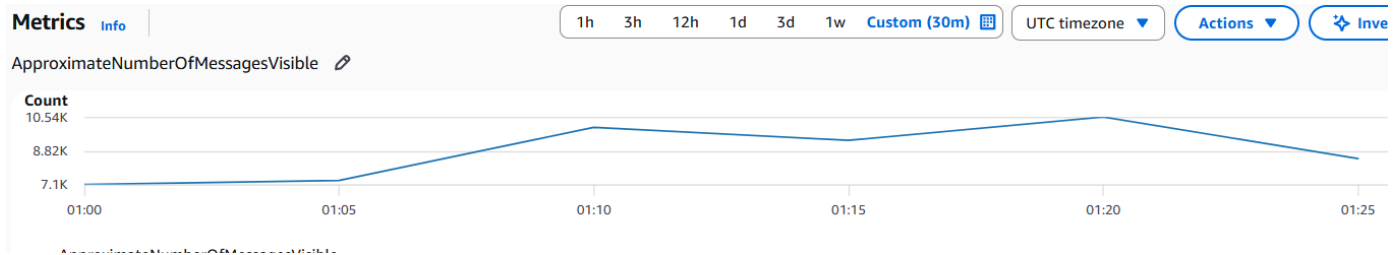
$$60 \times 3 = 180 \text{ workers}$$

So the minimum theoretical workers needed is:

180 worker goroutines

Whether that is practical in one tiny ECS task with only CPU 256 and Memory 512MB is another matter. So it is just the theoretical minimum, but in production you could likely scale out horizontally too.

CloudWatch Monitoring



Analysis Questions

- **How many times more orders did your asynchronous approach accept compared to your synchronous approach?**

In the flash-sale tests, the asynchronous approach accepted about 186.3 times more orders than the synchronous approach. The sync endpoint completed only 19 requests in 60 seconds, while the async endpoint accepted 3,540 requests in the same 60-second window. This happened because the async API returned immediately after publishing to SNS, while the synchronous version blocked on the 3-second payment bottleneck.

- **What causes queue buildup and how do you prevent it?**

Queue buildup happens when orders are accepted faster than they are processed. In this system, the API can acknowledge orders quickly, but the background processor only completes a limited number per second, so unprocessed messages accumulate in SQS.

- **When would you choose sync vs async in production?**

Synchronous processing is better when the client must know the final result immediately and the work is fast enough to complete within an acceptable response time. Asynchronous processing is better for slow or bursty workloads where fast acknowledgment matters more than immediate completion.

Next: Replace your ECS workers with Lambda functions, comparing container vs serverless architectures for the same workload.

HW7 > order-system > src > lambda_main.go > ...

```
14 func handler(ctx context.Context, event events.SNSEvent) error {
15     if coldStart {
16         log.Println("COLD START: first invocation in this execution")
17         coldStart = false
18     }
19
20     for _, record := range event.Records {
21         log.Printf("processing SNS message id=%s", record.SNS.MessageID)
22         time.Sleep(3 * time.Second)
23         log.Printf("completed SNS message id=%s", record.SNS.MessageID)
24     }
25
26     return nil
27 }
28
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

locust - tests + - [] [] ...

go get github.com/aws/aws-lambda-go/lambda

go get github.com/aws/aws-lambda-go/events

go mod tidy

GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -o bootstrap

lambda_main.go zip lambda.zip bootstrap

```
21 resource "aws_lambda_function" "order_processor" {
29     timeout          = 10
30 }
31
32 resource "aws_sns_topic_subscription" "orders_to_lambda" {
33     topic_arn = aws_sns_topic.orders.arn
34     protocol  = "lambda"
35     endpoint  = aws_lambda_function.order_processor.arn
36 }
37
38 resource "aws_lambda_permission" "allow_sns" {
39     statement_id = "AllowExecutionFromSNS"
40     action       = "lambda:InvokeFunction"
41     function_name = aws_lambda_function.order_processor.function_name
42     principal     = "sns.amazonaws.com"
43     source_arn    = aws_sns_topic.orders.arn
44 }
```

Apply Lambda infrastructure

terraform apply -auto-approve

-var="container_image=316009999564.dkr.ecr.us-east-1.amazonaws.com/ordersys:latest"

Scale ECS processor to 0 for cleaner Lambda-only testing

aws ecs update-service --cluster ordersys-cluster --service ordersys-processor
--desired-count 0 --region us-east-1

Send 5-10 orders through your existing Part II Order API:

curl -X POST

<http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com/orders/async> -H

"Content-Type: application/json" -d

'{"customer_id":123,"items":[{"item_id":1,"name":"shirt","quantity":1}]}'

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems/HW7/order-system/src (main)

```
$ curl -X POST http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com/orders/async -H "Content-Type: application/json" -d '{"customer_id":123,"items":[{"item_id":1,"name":"shirt","quantity":1}]}'
```

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems/HW7/order-system/src (main)

```
$ curl -X POST http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com/orders/async -H "Content-Type: application/json" -d '{"customer_id":123,"items":[{"item_id":1,"name":"shirt","quantity":1}]}'
{"message":"order accepted","order_id":"1773540413504516744","status":"pending"}
```

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems/HW7/order-system/src (main)

```
$ curl -X POST http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com/orders/async -H "Content-Type: application/json" -d '{"customer_id":123,"items":[{"item_id":1,"name":"shirt","quantity":1}]}'
{"message":"order accepted","order_id":"1773540415462921275","status":"pending"}
```

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems/HW7/order-system/src (main)

```
$ curl -X POST http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com/orders/async -H "Content-Type: application/json" -d '{"customer_id":123,"items":[{"item_id":1,"name":"shirt","quantity":1}]}'
{"message":"order accepted","order_id":"1773540417228903713","status":"pending"}
```

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems/HW7/order-system/src (main)

```
$ curl -X POST http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com/orders/async -H "Content-Type: application/json" -d '{"customer_id":123,"items":[{"item_id":1,"name":"shirt","quantity":1}]}'
{"message":"order accepted","order_id":"1773540419152103629","status":"pending"}
```

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems/HW7/order-system/src (main)

```
$ curl -X POST http://ordersys-alb-56403289.us-east-1.elb.amazonaws.com/orders/async -H "Content-Type: application/json" -d '{"customer_id":123,"items":[{"item_id":1,"name":"shirt","quantity":1}]}'
{"message":"order accepted","order_id":"1773540421079854077","status":"pending"}
```

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems/HW7/order-system/src (main)

o \$ []

Observe Cold Starts

Cold Starts

▶	Timestamp	Message
		No older events at this moment. Retry
▼	2026-03-15T02:09:46.677Z	INIT_START Runtime Version: provided:al2.v143 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:64c84fb4bae0287749677ef42dbfb9c0b2bf4a3c7b1a638630bcc30cf04c7b INIT_START Runtime Version: provided:al2.v143 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:64c84fb4bae0287749677ef42dbfb9c0b2bf4a3c7b1a638630bcc30cf04c7b

Warm Starts

▶	Timestamp	Message
▼	2026-03-15T02:06:51.186Z	REPORT RequestId: c7f9ccb6-4ee5-41e9-999d-163360cace78 Duration: 3003.48 ms Billed Duration: 3004 ms Memory Size: 512 MB Max Memory Used: 20 MB REPORT RequestId: c7f9ccb6-4ee5-41e9-999d-163360cace78 Duration: 3003.48 ms Billed Duration: 3004 ms Memory Size: 512 MB Max Memory Used: 20 MB
▼	2026-03-15T02:06:51.649Z	START RequestId: 4b601155-4275-43d2-b45e-f56fa1e38bef Version: \$LATEST START RequestId: 4b601155-4275-43d2-b45e-f56fa1e38bef Version: \$LATEST
▼	2026-03-15T02:06:51.649Z	2026/03/15 02:06:51 processing SNS message id=b2818462-a947-51fa-92da-e652bf3c4c32 2026/03/15 02:06:51 processing SNS message id=b2818462-a947-51fa-92da-e652bf3c4c32
▼	2026-03-15T02:06:54.649Z	2026/03/15 02:06:54 completed SNS message id=b2818462-a947-51fa-92da-e652bf3c4c32 2026/03/15 02:06:54 completed SNS message id=b2818462-a947-51fa-92da-e652bf3c4c32
▼	2026-03-15T02:06:54.650Z	END RequestId: 4b601155-4275-43d2-b45e-f56fa1e38bef END RequestId: 4b601155-4275-43d2-b45e-f56fa1e38bef
▼	2026-03-15T02:06:54.650Z	REPORT RequestId: 4b601155-4275-43d2-b45e-f56fa1e38bef Duration: 3001.61 ms Billed Duration: 3002 ms Memory Size: 512 MB Max Memory Used: 20 MB REPORT RequestId: 4b601155-4275-43d2-b45e-f56fa1e38bef Duration: 3001.61 ms Billed Duration: 3002 ms Memory Size: 512 MB Max Memory Used: 20 MB

How often do cold starts occur? (First request, after ~5+ minutes idle)

Not that often since there was only one.

What's the overhead? (73ms on 3000ms = 2.4% impact)

Does this matter for 3-second payment processing?

In this test, the cold start appeared only on the first invocation, while the remaining invocations were warm starts. The overhead appears small compared with the 3-second payment-processing delay, so it does not materially affect this workload. For long-running order-processing tasks, cold-start latency is noticeable but not operationally significant.

Cost Reality Check!

Your Current Part II Cost

2 ECS tasks × \$8.50/month = \$17 per month (always running)

Lambda Cost Calculator

AWS Lambda Pricing (official pricing):

\$0.20 per million requests

\$0.0000166667 per GB-second

Free tier: 1M requests + 400K GB-seconds monthly

**Example: 10,000 orders/month, 3s each, 512MB (0.5GB) Requests: 10,000 (under 1M free tier) = \$0 GB-seconds: $10,000 \times 3 \times 0.5 = 15,000$ (under 400K free tier) = \$0
Monthly cost = \$0 (FREE!)**

Break-even calculation: When does Lambda cost \$17/month?

Free tier covers: 1M requests + 400K GB-seconds

For 3s/0.5GB orders: $400K \div 1.5 = \sim 267K$ orders/month FREE

Beyond free tier: Need ~1.7M requests/month to reach \$17

Reality: Lambda is FREE for startups under 267K orders/month

For 10,000 orders/month, 3 seconds each, 512MB = 0.5GB:

Requests = 10,000

GB-seconds = $10,000 \times 3 \times 0.5 = 15,000$

This is under the Lambda free tier, so monthly cost is effectively:

\$0

For ECS comparison:

2 ECS tasks × \$8.50/month = \$17/month

Write one paragraph: Should your startup switch to Lambda? Why or why not?

For a startup with low or moderate order volume, I would recommend switching to Lambda because it removes most of the operational overhead of running ECS workers and managing SQS backlogs, while still supporting the same asynchronous design through SNS. In this workload, the 3-second payment processing time makes the additional cold-start overhead very small, and the monthly cost stays effectively free at low volume. However, if the business requires stronger delivery guarantees, better retry control, or queue-based buffering during traffic spikes, I would keep the ECS plus SQS design instead of fully switching to Lambda.

Is the cost advantage compelling? Yes. For 10,000 orders per month, Lambda is effectively free under the free tier, while the ECS design costs about \$17/month even when idle.

Can you accept losing SQS guarantees? Only if the startup can tolerate weaker delivery guarantees and less retry control. If order durability, replay, and controlled retries are important, the ECS + SQS design remains the safer production choice.

terraform destroy -auto-approve